Technical Report

Project Title

Habit & Productivity Analytics API

Submission Links

- GitHub Repository: [INSERT GITHUB LINK]
- API Documentation PDF: [INSERT API DOC LINK]
- Presentation Slides: To be submitted separately as part of the coursework package.

Section 1. Introduction and Project Overview

The Habit & Productivity Analytics API is a data-driven REST API that turns a 90-day habit CSV dataset into validated, reusable services. The project focuses on a **correct, testable, and demonstrable system** rather than only a prototype.

The final system provides end-to-end workflow coverage: CSV cleaning and mapping, relational storage, REST CRUD endpoints, backend analytics aggregation, and frontend dashboard consumption. It includes FastAPI, SQLAlchemy, JWT-protected write operations, `/analytics/summary`, and a fully passing pytest suite.

Section 2. System Architecture and Technology Choices

The system follows a layered design: routes (HTTP handling), schemas (validation contract), CRUD/business logic, ORM models, and database session management. Typical flow is client request → route → schema validation → CRUD → database → JSON response.

Technology choices were made for clarity and reproducibility: FastAPI (API framework/docs), SQLAlchemy (ORM), Pydantic (validation), Pandas (CSV preprocessing), and Pytest (automated verification). Database configuration is environment-based, using SQLite locally and PostgreSQL in deployment through `DATABASE_URL`.

Section 3. Data Source, Schema Mapping, and Import Pipeline

This project uses the **"90-Day Habit Tracker for Personal Growth"** dataset obtained from Kaggle (https://www.kaggle.com/datasets/uthaya1995/90-day-habit-tracker-for-personal-growth).
It was selected because it includes repeated daily records, mixed activity indicators, and realistic quality issues (for example missing or inconsistent values), making it suitable for preprocessing-driven API modelling.

CSV data is mapped to two entities: `habit_categories` (metadata) and `habit_records` (dated observations). For safe deployment, the project uses `seed_history` plus a guarded pre-deploy seed script to prevent duplicate imports.

Raw Field	Internal Field	Transformation Rule

Raw Field	Internal Field	Transformation Rule
`Date`	`record_date`	Parsed into ISO date format (`YYYY-MM-DD`); invalid values are skipped.
`Workout_Duration_Min`	`duration_minutes`	Converted to numeric minutes and validated as non-negative when present.
`Journaling (Y/N)`	activity signal	Normalised into a boolean-style signal for rule-based classification.
`Reading_Min`	activity signal	Converted to a numeric indicator and used in rule-based classification.
`Sleep_Hours`	activity signal	Converted to a numeric value and retained as an input signal where available.
activity signals	category	Mapped into categories such as Fitness, Health, or Wellness using explicit rules.
multiple fields	`completed`	Inferred through threshold-based and signal-based decision rules.

Rule-based transformation was intentionally chosen over opaque heuristics to maximise transparency, explainability, and reproducibility.

Section 4. API Design, CRUD Functionality, and Analytics

The API is centred on `habit_categories` and `habit_records`, both with full CRUD support. Endpoints follow REST conventions (`GET`, `POST`, `PUT`, `DELETE`) with consistent JSON responses and standard status behavior, including `204 No Content` for successful deletion.

Beyond CRUD, `/analytics/summary` returns six backend-computed metrics: `total_records`, `completed_records`, `completion_rate`, `average_duration`, `records_per_category`, and `daily_trend`. The frontend dashboard calls this endpoint on initial load and refresh, demonstrating end-to-end integration.

Interactive API documentation (Swagger UI)

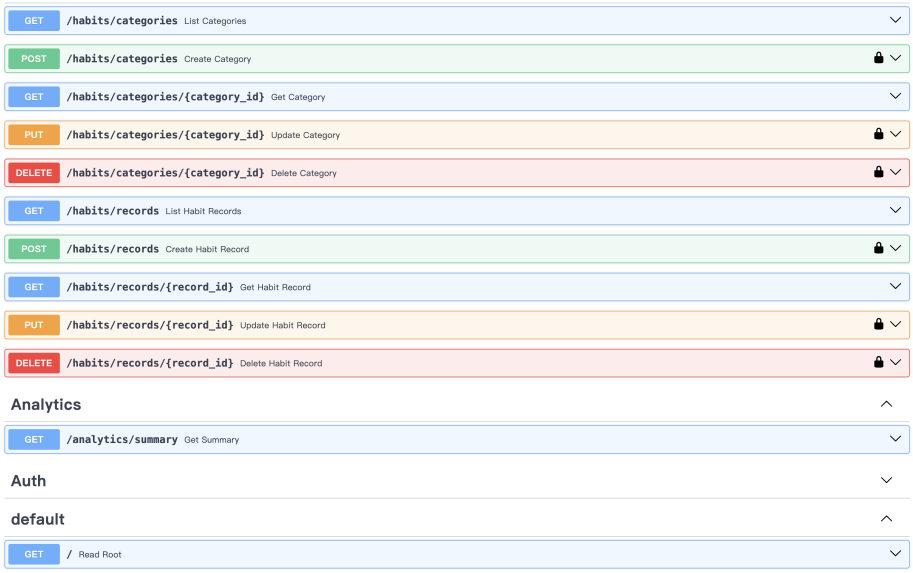

Figure 1. Interactive API documentation (Swagger UI) used for endpoint-level verification.

/analytics/summary response

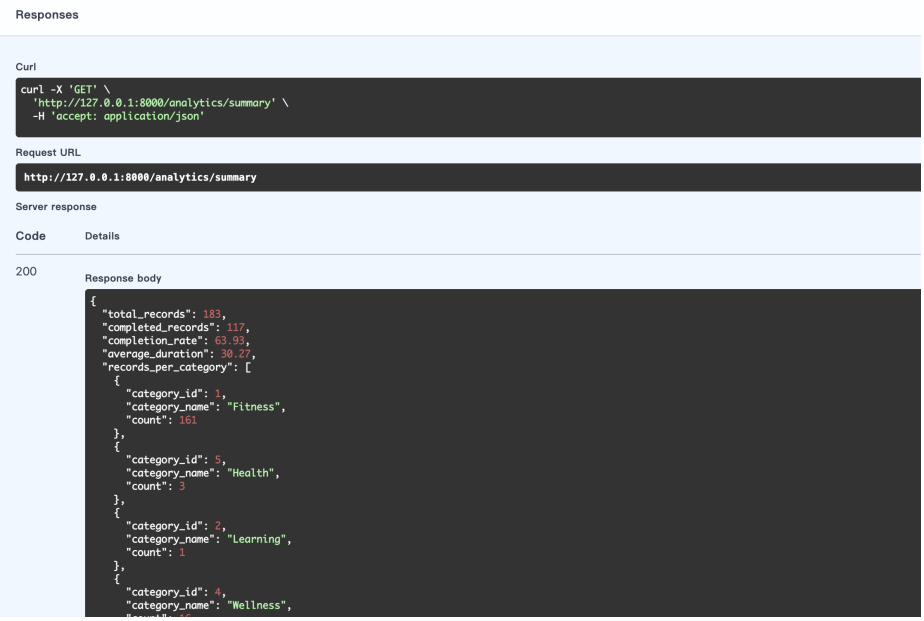

Figure 2. `/analytics/summary` response returned from the backend aggregation endpoint.

Section 5. Authentication, Error Handling, and Validation

Authentication uses JWT: clients authenticate via `/auth/login` (JSON) or `/auth/token` (OAuth2 form), then use `Authorization: Bearer <token>` for protected requests. Write operations for categories/records are protected, and `/auth/me` is also protected.

Error handling is standardised: `400` (business-rule errors), `401` (auth failures), `404` (not found), and `422` (validation). Responses follow a consistent `detail` + `code` structure. Validation is enforced through Pydantic schemas at request boundaries.

Section 6. Testing and Validation

Testing was carried out at API level using Pytest with FastAPI TestClient to verify both expected and failure-path behaviour. The current suite is fully passing.

> Test Execution Evidence:
>
> ```
> 22 passed in 0.84s
> ```

This provides reproducible evidence of correctness. Coverage includes authentication, authorization, CRUD flows, duplicate/invalid input handling (`400`), not-found cases (`404`), analytics response checks, and schema boundary validation (`422`). Runtime checks were also performed via Swagger UI and helper scripts.

Automated test execution result

Figure 3. Automated test execution output showing all tests passed.

Section 7. Challenges and Lessons Learned

Main challenges were data quality, schema mapping, and defining `completed` in a mixed-signal dataset. A transparent multi-signal rule was used to balance practicality and explainability.

Another key decision was analytics placement: computing summaries in the backend improved consistency, integrity, and testability compared with frontend-only calculation.

Section 8. Limitations and Future Improvements

Current limitations are intentional for coursework scope: lightweight demo authentication, minimal user management, summary-level analytics only, and no migration tooling yet.

Future production-oriented work includes stronger user and token controls, richer analytics filters/comparisons, migration support, observability, and CI/CD hardening.

Section 9. GenAI Declaration and Reflection

GenAI tools were used in a supporting role (ideation, debugging support, code refinement suggestions, and documentation drafting), not as a substitute for engineering judgement.

Tools Used and Purpose (Summary)

- **GitHub Copilot** was used during implementation to accelerate repetitive coding tasks, improve route and function structure, and reduce boilerplate.
- **ChatGPT** was used for design discussion, debugging analysis, and drafting or polishing documentation, including the README, API documentation, technical report, and GenAI declaration.

All key technical decisions were made by me and accepted suggestions were validated through code and runtime checks.

GenAI improved speed and clarity, but ownership did not shift; **final decisions remained under human control**.

Curated GenAI interaction summaries/logs are included as supplementary submission material (appendix) in line with coursework requirements.

Section 10. Appendix

This appendix lists supporting materials provided with submission.

- Curated GenAI interaction summaries/logs (supplementary material).
- Submission artefacts: source repository, API documentation PDF, technical report, and presentation slides (submitted separately).
- Evidence assets: Swagger UI screenshot, `/analytics/summary` screenshot, and pytest output screenshot.

Section 11. References

1. Uthaya, S. (n.d.) *90-Day Habit Tracker for Personal Growth*. Kaggle. Available at: https://www.kaggle.com/datasets/uthaya1995/90-day-habit-tracker-for-personal-growth (Accessed: 20 April 2026).
2. FastAPI (n.d.) *FastAPI Documentation*. Available at: https://fastapi.tiangolo.com/ (Accessed: 20 April 2026).
3. SQLAlchemy (n.d.) *SQLAlchemy 2.0 Documentation*. Available at: https://docs.sqlalchemy.org/ (Accessed: 20 April 2026).
4. IETF (2015) *RFC 7519: JSON Web Token (JWT)*. Available at: https://datatracker.ietf.org/doc/html/rfc7519 (Accessed: 20 April 2026).